



Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: Victor David Perez Argueta **Carné:** 1074325

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una pipeline (script) CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado

Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:

GitHub: <https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet>

El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

Realización de la actividad

1. Preparar el repositorio

En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable

2. Revisar la aplicación:

Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento.

Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>

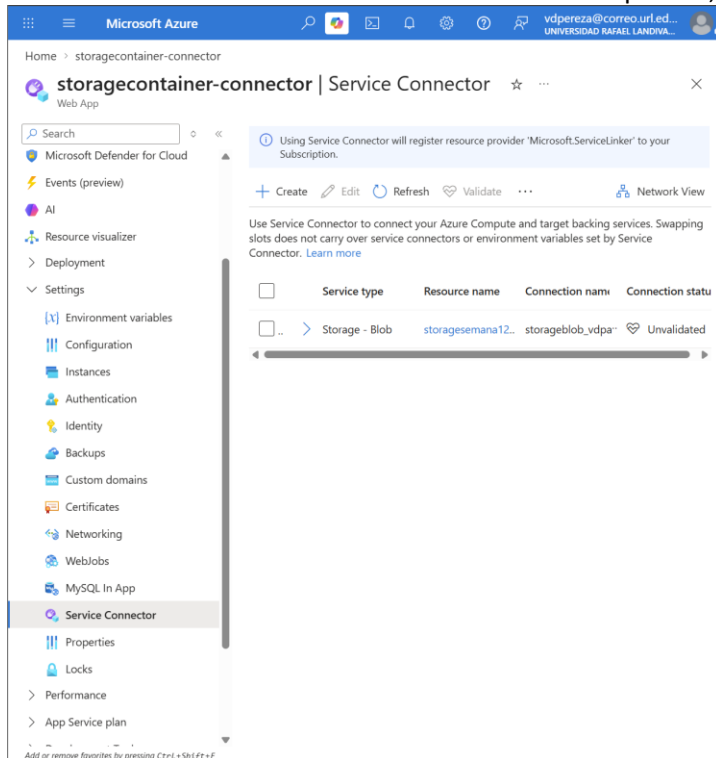


3. Crear DotNet Env

- En terminal, instalar: **dotnet add package DotNetEnv**
- Crear .env file en raíz de proyecto, valores ->
AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/
AZURE_STORAGE_CONTAINER_NAME=default
- Crear .env.production con los mismos valores en el mismo lugar
- Añadir cambio brindado en program.cs
- dotnet clean -> dotnet restore -> dotnet build -> dotnet run

4. Crear recursos en Azure

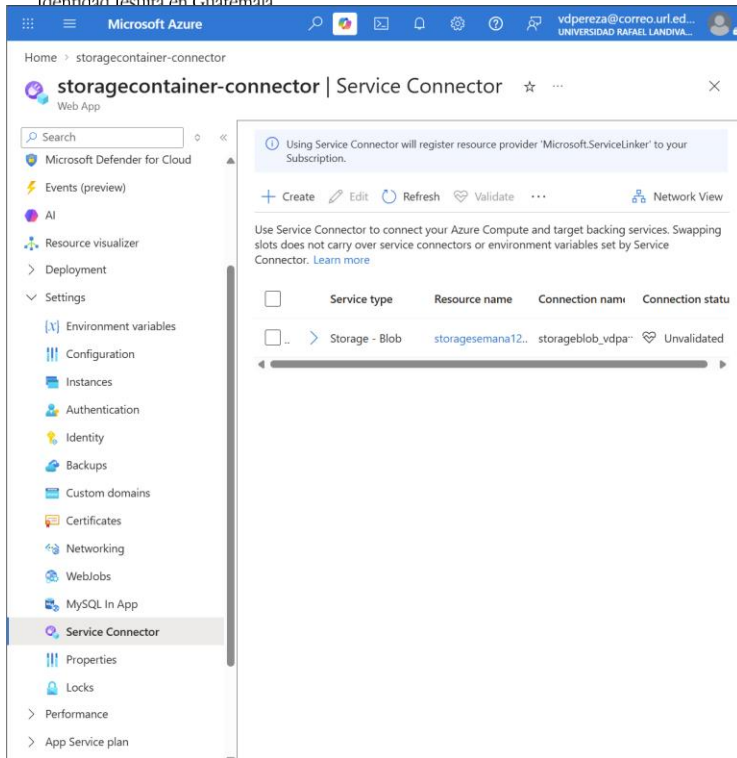
En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.



5. Conectar App Service con Blob Storage

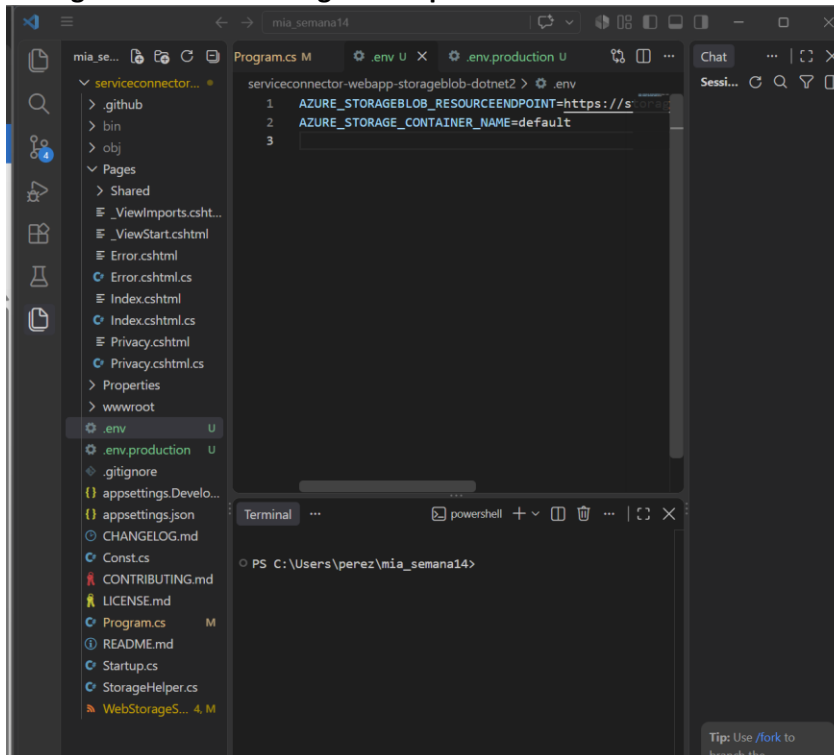
Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

- Azure Portal → **Storage accounts** → **tu Storage Account** → **Data storage** → **Containers**
- Crear contenedor "default"
- Activa identidad administrada en App Service: **Azure Portal** → **App Service** → **Settings** → **Identity** → **System assigned: on** → **Save** (web app creada semana 13)
- Configurar la conexión entre App Service y Blob Storage: **App Service** → **Settings** → **Service Connector** → **Create** → **Seleccionar "Storage Blob"** → **Seleccionar Storage Account [tu_blob_name]** → **Authentication "System-assigned managed identity"** → **Review + Create**.



e) Reemplazar storage blob endpoint y container nombre en .env file

Storage Account → Settings → Endpoints → Blob service



6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

- Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: `az --version`
- Inicie sesión en Azure: `az login`
- Verifique la cuenta de Azure activa: `az account show`



- e) Si se muestra un accessToken la autenticación esta correcta, ejecutar -> dotnet run
- f) Si hay error en acceso, ejecutar en terminal vs code: az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"

Obtener info: az account show --query "{subscriptionId:id, user:user.name}" -o table

az storage account show --name "tu_storage" --query "{id:id, resourceGroup:resourceGroup}" -o table

- g) Cerrar process ctr +c y logear de nuevo: az logout y az login, Por ultimo dotnet run

WebStorageSample Home Privacy

Hello Service Connector!
UTC Now: 9/10/2026 01:59:32
+00:00.

Learn about [building Web apps with ASP.NET Core](#).



×

+ Add Directory Upload Change access level Refresh ...

default

Authentication method: Access key (Switch to Microsoft Entra user account)

Add filter

Search blobs by prefix (case-sensitive)

Only show active blobs

Showing all 4 items

☐	Name	Last modified	Access tier
☐	 HelloWorld	8/10/2026, 20:04:35	Hot (Inferred)
☐	 prueba1.txt	8/10/2026, 20:09:09	Hot (Inferred)
☐	 prueba2.txt	8/10/2026, 20:09:09	Hot (Inferred)
☐	 prueba3.txt	8/10/2026, 20:09:09	Hot (Inferred)

Nota: no olvidar subir cambios a repo github fork

7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo "azure-pipelines.yml"

Colocar en el archivo lo siguiente:

trigger:

- main

pool:

vmImage: 'windows-latest'

variables:

buildConfiguration: 'Release'

dotnetVersion: '9.x'

Completar posteriormente

azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'

appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

1. Instalar .NET 9

- task: UseDotNet@2

displayName: 'Instalar .NET 9 SDK'

inputs:

packageType: 'sdk'

version: '\$(dotnetVersion)'

2. Restaurar dependencias

- task: DotNetCoreCLI@2



inputs:

command: 'restore'

projects: '**/*.csproj'

3. Configurar ambiente de producción

- powershell: |

Copy-Item ".env.production" ".env" -Force

displayName: 'Configurar .env de producción'

4. Compilar

- task: DotNetCoreCLI@2

displayName: 'Compilar aplicación'

inputs:

command: 'build'

projects: '**/*.csproj'

arguments: '--configuration \$(buildConfiguration) --no-restore'

5. Publicar

- task: DotNetCoreCLI@2

displayName: 'Publicar aplicación'

inputs:

command: 'publish'

projects: '**/*.csproj'

arguments: '--configuration \$(buildConfiguration) --output \$(Build.ArtifactStagingDirectory)/publish --no-build'

zipAfterPublish: true

6. Desplegar en Azure App Service

- task: AzureWebApp@1

displayName: 'Desplegar en Azure App Service'

inputs:

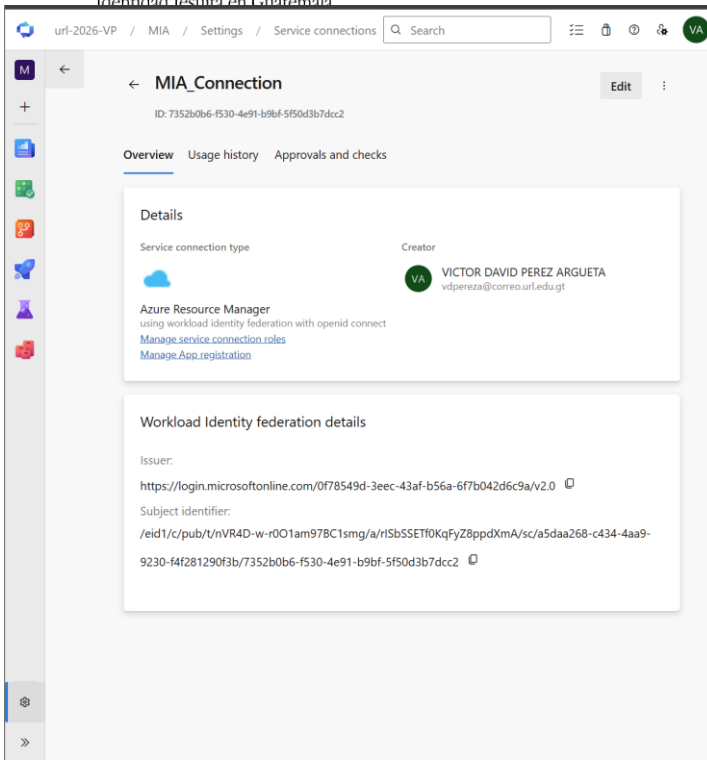
azureSubscription: '\$(azureSubscription)'

appType: 'webApp'

appName: '\$(appName)'

package: '\$(Build.ArtifactStagingDirectory)/publish/**/*.zip'

- Conectarse a Azure DevOps son misma cuenta que azure portal -> <https://aex.dev.azure.com/me>
- Crear pipeline: **En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml**
- Probar seleccionando "run"
- Conectar Azure DevOps con Azure Portal: **Project Settings → Service connections → New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre "MIA_Connection" -> Seleccionar "grant access permission to all pipelines" -> Save**



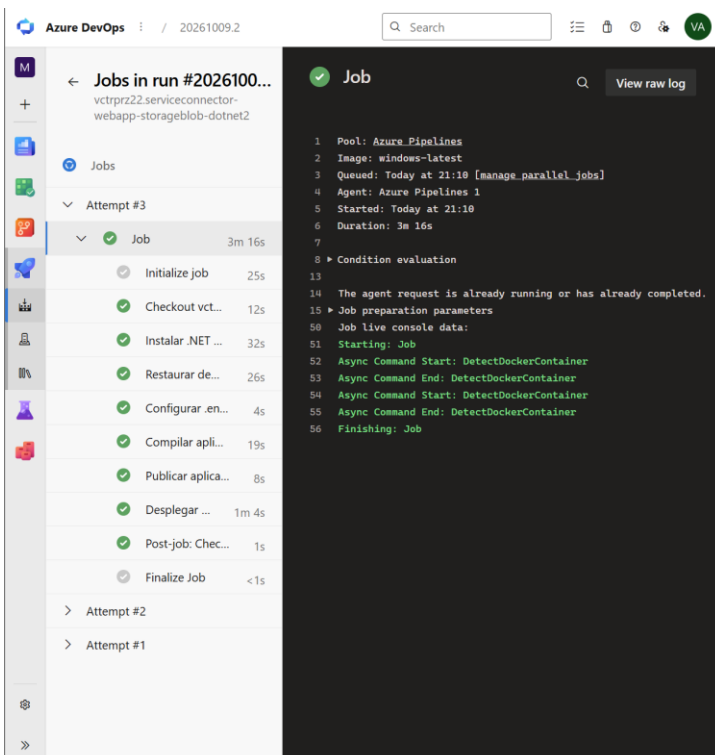
9. Conectar Pipeline con App

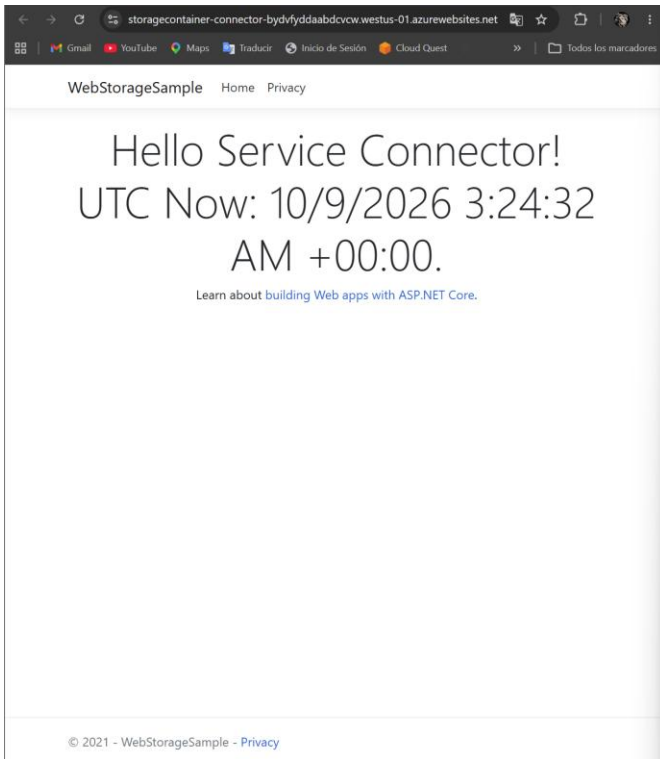
- Obtener nombre app: En Azure Portal → App Services → tu aplicación → Copia nombre app
- en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

azureSubscription: TU_Service_Connection_name

appName: Tu_app_name

- Al hacer commit, verificar el pipeline se ejecuta automáticamente





10. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

- acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings
-> colocar AZURE_STORAGEBLOB_RESOURCEENDPOINT
https://tu_storage.blob.core.windows.net/
(si existe obviar)
- Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica
- En la app publicada realizar cambios en archivos
- Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios



Microsoft Azure

vdpereza@correo.url.ed...
UNIVERSIDAD RAFAEL LANDIVA...

Home > storagecontainer-connector

storagecontainer-connector | Environment variables

Web App

Search

Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Microsoft Defender for Cloud
Events (preview)
AI
Resource visualizer
Deployment
Settings
Environment variables
Configuration
Instances
Authentication
Identity
Backup

App settings

Connection strings

Search

+ Add

Refresh

Show values

...

Name	Value	Deployment
AZURE_STORAGEBLOB_RESOUR...	Show value	✓
WEBSITE_ENABLE_SYNC_UPDA...	Show value	
WEBSITE_RUN_FROM_PACKAGE	Show value	



11. Preguntas de analisis

1. ¿Para qué sirve el archivo .env en la aplicación?

El archivo .env sirve para guardar datos de configuración que la aplicación necesita para funcionar, como la dirección del servicio de almacenamiento y el nombre del contenedor. De esta forma, estos valores no tienen que escribirse directamente dentro del código del programa.

2. ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?

Blob Storage permite guardar los archivos en la nube de forma independiente a la aplicación. Esto evita que los archivos se pierdan cuando el proyecto se actualiza, se vuelve a publicar o se ejecuta desde otro equipo. También facilita que los archivos puedan mantenerse disponibles sin depender directamente del proyecto.

3. ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?

El endpoint de Blob Storage es la dirección que utiliza la aplicación para conectarse al servicio donde se almacenan los archivos.

La URL de App Service, en cambio, es la dirección pública que permite acceder a la aplicación desde un navegador.

Por lo tanto, uno se utiliza para acceder al almacenamiento y el otro para acceder a la aplicación.

4. ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?

La identidad administrada permite que Azure reconozca a la aplicación y le otorgue los permisos necesarios sin tener que guardar claves de acceso dentro del código.

Esto ayuda a mejorar la seguridad, ya que evita exponer información sensible como una Storage Account Key y permite controlar los permisos directamente desde Azure.

5. ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?

GitHub se utiliza para almacenar el código del proyecto y guardar los cambios realizados.

Azure DevOps se encarga de automatizar el proceso de preparación y publicación de la aplicación mediante el pipeline.

La Service Connection permite que Azure DevOps tenga autorización para conectarse con los recursos de Azure, por ejemplo el App Service, y realizar el despliegue de la aplicación.

6. ¿Qué ocurre después de hacer push a main?

Cuando se realiza un `push` a la rama `main`, Azure DevOps detecta automáticamente que hubo un cambio en el repositorio.

Después se ejecuta el pipeline configurado, el cual restaura las dependencias, compila el proyecto, prepara la aplicación y finalmente la despliega en Azure App Service.

De esta forma, los nuevos cambios realizados en el proyecto pueden quedar publicados automáticamente.

7. Si el pipeline termina correctamente pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?

Primero revisaría los permisos que tiene la aplicación sobre el Blob Storage.

También comprobaría que la identidad administrada del App Service tenga asignado un rol adecuado, como Storage Blob Data Contributor

Además, verificaría que el endpoint del Storage Account y el nombre del contenedor sean correctos.

Un error 403 normalmente indica que la aplicación sí logró comunicarse con Azure, pero no tiene los permisos necesarios para realizar la operación.



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-14/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.